# Backend Engineering Thinking Test

Architecture first. Then event processing, concurrency, idempotency and failure recovery.

## About NavSecura

NavSecura is building a security-intelligence platform for businesses that already use CCTV cameras, alarms and other security systems. Traditional CCTV systems mainly record video or generate individual alerts. NavSecura aims to receive observations from different security sources, understand what may be happening, combine related observations, preserve evidence, and help an operator make a safe response decision.

A simplified example could look like:

```
Camera 1 -> person detected
Camera 2 -> person detected
Door sensor -> door opened
Customer -> panic request
```

These should not necessarily become four unrelated alerts. They may be different pieces of evidence about one operational situation.

**The real NavSecura platform is significantly larger than this exercise. This task is intentionally small. We are primarily interested in how you think.**

## Part 1 - Architecture Thinking

Before writing code, design a small architecture for the following system.

- NavSecura has many customers, many sites, many cameras/sensors, a backend API, a database, background processing, and an operator application.
- Security sources continuously send events such as PERSON_DETECTED, VEHICLE_DETECTED, DOOR_OPENED, CAMERA_OFFLINE, PANIC_REQUEST and SMOKE_DETECTED.
- The system should receive and preserve events, associate related events where appropriate, process them asynchronously, produce an operational case or recommendation, and survive retries and failures.

Draw a small architecture diagram. It does not need to look professional; a hand drawing is completely acceptable. Show the major components and the direction in which information travels.

Then explain:

- Why you selected those components.
- Where the source of truth lives.
- What happens between receiving an event and showing something to an operator.
- What happens if part of the system becomes unavailable.
- What you think should happen when thousands of events arrive together.

Do not try to design the entire NavSecura product. Keep the architecture small enough that you could realistically implement it.

## Part 2 - Event Processing Exercise

You are now building a simplified version of that architecture.

Assume the system stores events similar to:

```
event_id
customer_id
site_id
source_id
event_type
observed_at
```

A background worker processes those events and creates or updates an operational case.

For example:

```
12:00:01 Camera A PERSON_DETECTED
12:00:03 Camera B PERSON_DETECTED
12:00:05 Door Sensor DOOR_OPENED
```

These events belong to the same site and may represent one activity.

Your task is to design and implement the important parts of this event-processing system.

You may choose a reasonable backend language/framework that you are comfortable with. Use a relational database such as PostgreSQL.

### Problem A - Duplicate Event

A camera sends an event such as EV-10001 / PERSON_DETECTED. Because of a network retry, the same event may arrive again and again, possibly at exactly the same time through different backend instances.

The system must not treat the same logical event as multiple different events. Show how you would handle this.

### Problem B - Many Events at the Same Time

Assume many cameras and sensors send events simultaneously. For example, 10,000 events arrive in a short burst. Some are unrelated, some belong to the same site, and some may describe the same activity.

Your system must continue accepting events without corrupting state. Explain and demonstrate how your architecture behaves.

We are interested in:

- events received
- events stored
- events processed
- duplicate events
- work waiting
- work completed

Do not optimise only for impressive throughput numbers. Correct accounting is more important.

### Problem C - Concurrent Workers

Assume several workers process events at the same time. Two workers must not accidentally process the same piece of work in a way that produces conflicting authoritative state.

Design the worker-processing mechanism. Demonstrate what happens when multiple workers run concurrently.

### Problem D - Worker Crash

Consider this sequence:

```
Worker receives work
↓
Worker processes the event
↓
Worker updates the case
↓
WORKER CRASHES
↓
The system never records that the work finished
```

Eventually another worker receives the same work. What should happen? Implement or demonstrate your solution.

> **The important requirement is that the event must not disappear, and the retry must not corrupt or unnecessarily duplicate the case.**

### Problem E - Related Events

Implement a simple rule for deciding whether new events belong to an existing case or should create a new case. You do NOT need machine learning.

For example, you may consider things such as customer, site, time, source, event type and an existing active case.

We are interested in your reasoning more than the exact correlation algorithm. Explain the limitations of your approach.

## Part 3 - Failure Scenario

After you have implemented your design, answer this scenario.

The database successfully saves an event. Immediately afterward, the backend process crashes. The client does not know whether the request succeeded, so it sends the same event again.

Explain exactly what happens in your implementation.

Then consider another case:

```
worker updates a case successfully

worker crashes

same work gets processed again later
```

Explain exactly what happens.

## What to Submit

Please provide:

- Your architecture diagram
- Your explanation of the architecture
- Source code

- Database schema/migrations
- Tests
- Instructions for running the project

Also include a short note answering:

- What was the hardest part of this problem?
- What assumptions did you make?
- What can still go wrong?
- If this system became 100 times larger, what would you change first?
- What would you monitor in production?
- What would you deliberately not build yet?

You may use normal engineering tools, documentation and AI assistants. If you use AI, that is completely acceptable.

We care about whether you understand the final design and can explain your decisions. After submission we will discuss the architecture and code together.